Department of Computer Science &
Engineering
The University of Texas at Arlington

Project Charter
CSE 4316 / CSE 4317: Senior Design
Summer 2026

AgriHome project logo: leaf and camera lens motif for vision-based agricultural monitoring.

AgriHome

GOVINDA AWASTHI
LAXMAN BHUSHAL
THUC DANG
SAGAR KHADKA
MUHAMMAD SAAD MEMON
ALEX TABLIZO
NATHAN NGUYEN TRAN

Revision History

Revision	Date	Author(s)	Description
0.1	02.18.2026	MM	Document creation
0.2	02.2026	Team	Complete draft
0.3	02.2026	Team	Release candidate 1
1.0	03.2026	Team	Official release
1.1	03.2026	Team	Customer change requests
2.0	07.23.2026	Team	Summer update: Pi/Moonraker edge + Vision Console
2.1	07.28.2026	Team	Klipper edge: Agri-Home/klipper + fswebcam; klipper_url

Contents

List of Figures

List of Tables

1 Problem Statement

Plant health management remains a significant challenge for farmers, gardeners, and greenhouse operators. Plant disease often goes undetected in early stages as symptoms may be unseen, mistaken for deficiencies, or environmental stress. Accurate diagnosis usually requires specialized knowledge, and many individuals cannot identify plant health issues efficiently. Incorrect identification may result in infection spread, reduced crop yield, and financial loss.

There is a need for a reliable system that provides early detection and continuous observation of plant health. Accurate disease identification would allow users to respond more effectively, encourage a healthier growing environment, and minimize crop damage. Addressing this issue improves plant health management while supporting informed decision-making in both large-scale facilities and small-scale home gardening environments.

Through Summer 2026, AgriHome has narrowed the practical gap between *manual inspection* and *always-on monitoring* by pairing a Vision Console web application with edge hardware (Raspberry Pi capture rigs) that can register themselves, report online status, and deliver tray imagery into a durable PostgreSQL-backed store for operator review and optional computer-vision analysis.

2 Methodology

AgriHome is a plant-health monitoring system built around continuous observation and early issue detection. The system captures plant imagery on a regular basis—via operator-initiated Take Picture commands, capture schedules destined for `raspberry-pi-edge`, or manual uploads—and analyzes images to detect irregularities when a computer-vision backend is configured.

The implemented stack through Summer 2026 includes:

- **Vision Console** — a Next.js App Router progressive web application with responsive desktop sidebar and mobile bottom navigation.
- **Identity** — Firebase Authentication (email/password and Google) with server-verified session cookies for protected routes and APIs.
- **Data plane** — PostgreSQL as the system of record for trays, plants, captures, schedules, edge devices, commands, and monitoring events; local file storage under configurable `STORAGE_*` roots served via `/api/files/...`.
- **Edge capture** — Raspberry Pi devices running Agri-Home/klipper (camera macros / `fswebcam` via `save_image.sh`) plus the `agrihome_agent` package (still hosted under the Agri-Home/moonraker repo companion): auto-provisioning, heartbeat, multipart ingest, and command claim/complete for capture workflows.
- **Optional CV** — a `cv-backend` species/leaf classifier reached via `CV_SPECIES_INFERENCE_URL`, with optional auto disease detection after Pi capture when enabled.

Using image recognition and machine learning when available, the system evaluates plant conditions and surfaces potential issues based on visual symptoms. When irregularities are detected, the Console presents status, reports, and monitoring events so operators can act before conditions worsen. The monitoring process is continuous rather than one-off manual inspection.

3 Value Proposition

AgriHome offers value by combining accessible computer vision with an operable edge-to-cloud workflow that greenhouse and lab users can run without becoming ML or embedded specialists.

For agricultural stakeholders, early identification of plant health issues reduces financial risk and improves crop quality. For university and academic stakeholders, the project integrates computer science, embedded systems, and agricultural applications, strengthening applied research and demonstration capacity. The Summer 2026 edge integration specifically reduces operator toil: devices self-provision into trays, report heartbeat/online status, and deliver frames through a secure device API key path rather than ad-hoc file transfer.

By investing funding, time, and expertise, stakeholders support innovation in sustainable agriculture, advance applied research, and contribute to a solution with meaningful economic and environmental impact.

4 High-Level Requirements

The following capabilities define Summer 2026 scope at charter level (detailed in the SRS):

- **Vision Console** — authenticated Progressive Web App (Next.js App Router) for dashboard, trays, plants, Devices list, and tray Raspberry Pi panel actions.
- **Identity** — Firebase Authentication with server-verified session cookies for human operators; device identity remains separate from Firebase users.
- **Persistence** — PostgreSQL as system of record for trays, plants, captures, schedules, `edge_devices`, device commands, and monitoring events; image files under `STORAGE_*` served via `/api/files/...`.
- **Edge provisioning** — Raspberry Pi agents register with CPU serial, MAC, hostname, and `DEVICE_PROVISIONING_SECRET`; AgriHome creates an `edge_devices` row, hashed API key (plaintext shown once), and a linked tray.
- **Heartbeat & commands** — agents report online status and claim pending commands (including `capture_now`) on a short interval.
- **Take Picture** — primary path queues agent capture (`fswebcam` / `camera-macros/save_image.sh` on Agri-Home/klipper); optional server-side HTTP streamer still when stored `klipper_url` is reachable, then multipart ingest.
- **Ingest & attach** — authenticated multipart upload creates `camera_captures`; plants may auto-attach to the linked tray; optional disease/species detection after Pi capture when `cv-backend` / `DEVICE_AUTO_VISION_ON_INGEST` is configured.
- **Connectivity** — Pi agents reach the AgriHome cloud URL over the network (LAN or Cloudflare tunnel / public HTTPS as needed); optional `klipper_url` streamer bases may use a tunnel when the AgriHome host cannot reach the Pi LAN for server-side stills.
- **Security** — rotatable provisioning secret, hashed device keys, revoke/rotate, ingest rate limits and size caps (see `docs/ops/EDGE_DEVICE_SECURITY.md`).

5 Development Milestones

- Project Charter first draft — February 2026
- System Requirements Specification — March 2026
- Architectural Design Specification — April 2026
- Demonstration of AI detection and test rig — April 2026
- Vision Console (Next.js) + PostgreSQL + Firebase auth baseline — Spring 2026
- `cv-backend` species classifier training/serving integration — Spring–Summer 2026
- **Hardware & Software Integration (Pi) sprint** — Summer 2026 (auto-provision `edge_devices`, device API keys, heartbeat, Take Picture, multipart ingest, optional disease detection after Pi capture, plant auto-attach to tray)
- Edge agent reference package (`agrihome_agent`; Klipper/fswebcam capture) — Summer 2026
- Detailed Design Specification (CSE 4317) — Summer 2026
- Demonstration of AI improvements and User Interface — TBA
- Demonstration of web server, edge, and AI integration — TBA
- CoE Innovation Day poster presentation — TBA
- Final Project Demonstration — TBA

6 Background

Plant health monitoring remains a persistent challenge across home gardening, greenhouse cultivation, and small- to medium-scale agricultural operations. Many plant species are susceptible to diseases, pest infestations, and nutrient deficiencies that initially manifest as subtle visual symptoms such as discoloration, spotting, wilting, or irregular leaf texture. Early-stage symptoms are often misinterpreted as environmental stress, leading to delayed intervention and crop loss.

The status quo relies heavily on manual visual inspection and personal experience. Accurate diagnosis frequently requires plant-pathology expertise that is not readily accessible. Even when symptoms are recognized, identifying the cause and determining corrective actions can be difficult. This gap results in inefficient pesticide use, improper fertilization, and avoidable yield reduction.

Advances in computer vision and machine learning provide a viable foundation for addressing this gap. Deep learning-based image recognition has demonstrated strong performance in plant disease classification. However, many solutions focus on classification accuracy alone and do not provide an end-to-end path from physical capture hardware to an operator dashboard with durable storage, schedules, and device lifecycle management.

AgriHome bridges that gap by integrating (1) edge capture via Raspberry Pi + Agri-Home/klipper (`fswebcam`), (2) authenticated Vision Console workflows for trays and plants, (3) optional leaf/species classification with confidence and recommendations, and (4) actionable monitoring events. The team does not currently have a direct industrial sponsor; Professor Noor sponsors the senior-design effort. The system is designed for home gardeners, greenhouse operators, and educational agricultural programs, aligning with the university's emphasis on applied AI and sustainable technology.

7 Related Work

Automated plant disease detection using image recognition has been widely studied. Convolutional Neural Networks (CNNs) have demonstrated strong performance classifying plant diseases from leaf images. Mohanty et al. showed that deep learning models trained on the PlantVillage dataset could achieve high accuracy in multi-class plant disease classification, including tomato leaf diseases, and that transfer learning with pretrained CNN architectures improves performance versus traditional machine learning [1].

Subsequent research compared architectures such as ResNet, VGG, MobileNet, and EfficientNet for tomato leaf disease detection, often reporting accuracies exceeding 95% under controlled dataset conditions [2]. Attention mechanisms and transformer-based models further improve fine-grained recognition under varying image conditions. Leaf segmentation (U-Net, Mask R-CNN) isolates plant tissue from background noise prior to disease analysis [3].

Commercial mobile applications such as Plantix allow users to upload plant images and receive predicted disease labels. Many commercial tools operate as black-box systems with limited transparency regarding confidence and next-step guidance, and they typically assume the user already has a phone camera in hand rather than an always-on tray fixture.

On the hardware side, AgriHome builds on the Agri-Home/klipper fork: G-code/macros for axis motion and `camera-macros/save_image.sh` (`fswebcam`) for local stills. An optional LAN HTTP streamer URL may still be stored for a server-side Take Picture fast path. The reference `agrihome_agent` remains under the Agri-Home/moonraker companion repo but speaks `klipperUrl` and fswebcam. This shortens integration time while separating motion/pose control from image ingest.

AgriHome differentiates itself by unifying leaf/species classification (when configured), structured recommendations, and a provisioned edge device lifecycle (register, heartbeat, secure ingest, Take Picture) into one Vision Console product.

8 System Overview

The solution is a camera-based plant monitoring system that observes plants arranged on trays or racks. Edge devices (Raspberry Pi + Agri-Home/klipper + `fswebcam`) capture images; AgriHome stores imagery and domain records; the Vision Console presents health status, history, and alerts. Optional CV services classify and report on captures.

At a high level, the Summer 2026 flow is:

1. **Provision:** Pi agent (`agrihome_agent`) registers at `/api/raspberry-pi/register` with CPU serial + `DEVICE_PROVISIONING_SECRET` (model default `raspberry-pi-klipper`); AgriHome creates an `edge_devices` row, SHA-256 hashed API key, and linked tray.

2. **Heartbeat:** Agent reports online; Console shows device status; pending commands may be claimed (`klipperUrl`; deprecated `moonrakerUrl` alias accepted).

3. **Capture:** Operator Take Picture primarily queues `capture_now` for the agent (`fswebcam` / `save_image.sh`). Optional server-side HTTP streamer still when `klipper_url` is reachable. Schedules may target `raspberry-pi-edge`.

4. **Ingest:** Multipart upload authenticates with `X-Agrihome-Device-Key`; originals land under `STORAGE_*` and `camera_captures` rows are created; plants may auto-attach to the tray.

5. **Optional CV:** When configured, disease/species detection may run after Pi capture via `cv-backend`.

6. **UI:** Dashboard, trays, plants, Devices nav, and tray Raspberry Pi panel (Take Picture, Klipper/streamer URL via `updateKlipperUrl`, link/revoke) surface results.

7. **Ops networking:** Bench Pi→cloud uses the AgriHome base URL (often via Cloudflare tunnel or HTTPS); optional server-side stills need a LAN-reachable or tunneled `klipper_url` (WARP local-network override when applicable). See `docs/ops/RASPBERRY_PI_KLIPPER.md`.

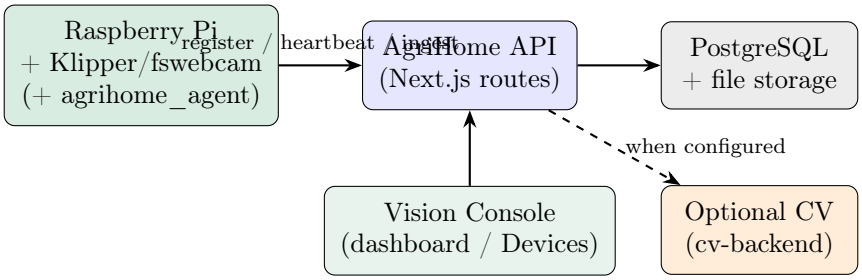

Figure 1: Summer 2026 system overview: Pi+Klipper to AgriHome API to storage/DB and dashboard, with optional CV.

```
   +------------------+
   |  Vision Console  |
   |  (Next.js PWA)   |
   +--------+---------+
            |
            v
   +----------------------+
   | AgriHome Next.js API |
   | /api/raspberry-pi/*  |
   | /api/devices/* ...   |
   +----------+-----------+
         |           |
         v           v
+-------------+  +----------------+
| PostgreSQL  |  | STORAGE_* files |
| trays/plants|  | /api/files/...  |
| edge_devices|  +----------------+
+-------------+
         ^
         |  optional
         v
+------------------+     +----------------------+
| cv-backend       |     | Pi + Klipper         |
| species classify |<--->| agrihome_agent       |
+------------------+     +----------------------+
```

9 Roles & Responsibilities

Stakeholders want a compact plant monitor that can detect diseases and reduce constant manual inspection. The sponsor is Professor Noor.

Key team roles (rotated across sprints as directed by course practice):

- **Scrum master** — backlog facilitation and sponsor contact; expected to also contribute to another technical role.
- **Hardware designers/programmers** — plant rack, camera fixture, Raspberry Pi / AgriHome/klipper bench setup (fswebcam), and edge agent bring-up.
- **Software programmers** — Vision Console UI, Next.js API routes, PostgreSQL schema/ migrations, and Firebase session integration.
- **AI designers** — `cv-backend` training/serving, species inference wiring, and evaluation of post-capture detection quality.

Roles are not permanent for the duration of the project. Members may hold multiple roles. The scrum master role rotates at the end of each sprint at the request of Professor Noor. Assignments follow expertise, so people often return to similar task clusters across sprints.

10 Cost Proposal

The budget allotted by the customer is $800, sourced from the CSE department of UTA. Funds cover image-capture hardware (including Raspberry Pi / webcam components as needed), compute for running models, materials for the testbed, and incidental implementation costs. University-provided server capacity continues to offset hosting costs where available.

10.1 Preliminary Budget

Item	Estimated Cost
Camera / webcam module	$70
Computer (local / Pi-class edge)	$60–$120
Computer (server)	$0 (provided by UTA where available)
Camera fixture	$50
Plant fixture / tray materials	$100
Plants	$50
Misc. networking / mounting	$50

Table 1: Preliminary budget (updated for edge capture components).

10.2 Current & Pending Support

The primary funding source is the CSE Department of The University of Texas at Arlington. The default amount of $800 should cover projected costs. If additional funding is required, Professor Noor has indicated willingness to provide support, though no exact amount or formal commitment has been made.

11 Facilities & Equipment

Most equipment needed is available at UTA. Construction of the plant rack and camera fixture uses makerspaces (Nedderman Hall and Library), including saws and 3D printers; CNC or laser cutting may be used if the fixture design requires it.

AI model training and deployment need GPU-capable machines. Senior Design lab computers may suffice subject to availability; otherwise team machines or containerized `cv-backend` workflows are used. External rented GPUs remain a fallback but are not required for the current scope.

Edge integration requires bench networking so (1) Pi agents can reach the AgriHome API (LAN, Cloudflare tunnel, or other HTTPS endpoint) and (2) optional HTTP streamer URLs on `klipper_url` are reachable when server-side stills are used when using the server-side Take Picture fast path (often `http://PI_IP:7125` API and/or nginx `:80` webcam stills, or a tunnel URL when the host cannot see the Pi LAN). Stable temperature and electrical supply are required for plant care; the Senior Design lab or Engineering Research Building balcony spaces remain candidate storage/test sites.

12 Assumptions

- Sufficient labeled leaf disease images (e.g., PlantVillage-style) remain available for training and evaluation of the classifier.
- Raspberry Pi + Agri-Home/klipper + `fswebcam` hardware is available for the Hardware & Software Integration (Pi) sprint.
- Team members have consistent access to computing resources for Next.js development, PostgreSQL, and model training/serving.
- Lab spaces remain available during normal operating hours for setup and testing.
- The image recognition model can achieve acceptable classification performance (e.g., >80% validation accuracy) within the planned training period when CV is in scope.
- Lighting during capture is sufficiently controlled to avoid extreme image-quality variability.
- A shared `DEVICE_PROVISIONING_SECRET` can be distributed securely to bench devices.
- Pi agents can reach AgriHome over LAN or via Cloudflare tunnel / public HTTPS; operators can set a reachable `klipper_url` (LAN IP or tunnel) for optional server-side stills.
- Course schedule and instructor expectations remain consistent with the projected sprint timeline (CSE 4316 historically; CSE 4317 continuation in Summer 2026).

13 Constraints

- Prototype demonstrations must complete within the senior-design course calendar (Spring / Summer 2026 milestones).
- Total development costs must not exceed the allocated $800 budget unless additional sponsor funding is approved.
- Development follows the sprint schedule defined by the course syllabus.

- The system must operate using consumer-grade cameras/webcams and computing hardware (including Raspberry Pi-class devices).
- The solution must comply with university IT and lab usage policies.
- Only publicly available datasets or self-collected images may be used for model training.
- Team size and available development hours are fixed by course enrollment and academic commitments.
- Device registration remains closed unless `DEVICE_PROVISIONING_SECRET` is configured; plaintext device API keys are shown only once at register/rotate.

14 Risks

Risk exposure is calculated as Exposure = Probability × Loss (in days).

Risk description	Prob.	Loss	Exposure
Insufficient model accuracy for reliable diagnosis	0.40	15	6.0
Limited availability of labeled disease images	0.35	12	4.2
Hardware malfunction or camera/webcam failure	0.25	8	2.0
Unexpected delays in model training or debugging	0.30	10	3.0
Team member availability conflicts	0.30	7	2.1
Image quality variability causing inconsistent results	0.35	9	3.15
Pi unreachable from AgriHome host (NAT / WARP LAN)	0.30	6	1.8
Provisioning secret leakage or stolen device API key	0.20	10	2.0

Table 2: Overview of highest-exposure project risks (Summer 2026 update).

Mitigation strategies include early dataset validation, backup hardware planning, incremental model testing per sprint, primary agent `fswebcam` capture with optional HTTP streamer fast path, hashed device keys with revoke/rotate, ingest rate limits, and documented LAN/WARP override guidance for bench networks.

15 Documentation & Reporting

15.1 Major Documentation Deliverables

The Project Charter defines project scope, objectives, stakeholders, and high-level risks. It is reviewed at major milestone checkpoints and updated when scope, schedule, or risk changes occur.

15.1.1 Project Charter

Maintained as a controlled document defining vision, scope, objectives, stakeholders, and high-level risks. Reviewed at the end of major milestones or sprints. Updates occur for significant scope, stakeholder, or risk adjustments. Major revisions are version-tracked and approved by the team.

Initial version delivery: Sprint 1. *Final version delivery:* Prior to final project submission (this document is Revision 2.1, Summer 2026 updated release).

15.1.2 System Requirements Specification

Living document reflecting agreed functionality and constraints. Formally reviewed at the beginning and end of each sprint. Updates occur when requirements are approved, modified, or clarified from customer feedback, sprint reviews, testing, or scope adjustments. Major requirement changes require team discussion and approval with version tracking.

15.1.3 Architectural Design Specification

Describes high-level structure, major components, interfaces, and data flow. Reviewed each sprint; updated for significant architectural changes (new subsystems, integration approaches). Companion ADS for the Vision Console layered architecture remains the Spring 2026 design baseline.

15.1.4 Detailed Design Specification

Describes internal design of modules, workflows, data structures, and algorithms. Updated throughout development and reviewed each sprint. The CSE 4317 Summer 2026 DDS reflects the real Next.js / PostgreSQL / Raspberry Pi implementation rather than earlier fictional schema drafts.

15.2 Recurring Sprint Items

15.2.1 Product Backlog

Prioritized list of features and technical work (including edge stories AGRI-101–AGRI-110). Refined continuously; ordered by customer value and risk.

15.2.2 Sprint Planning

Team selects backlog items for the sprint capacity, clarifies acceptance criteria, and identifies dependencies (hardware bench time, CV model availability, networking).

15.2.3 Sprint Goal

Single concise objective for the sprint (example Summer 2026: “Pi registers, heartbeats, and Take Picture lands a frame in the Console”).

15.2.4 Sprint Backlog

Committed tasks for the sprint, owned by individuals, tracked to done.

15.2.5 Task Breakdown

Stories decomposed into implementable tasks (API route, migration, UI panel, agent command, docs).

15.2.6 Sprint Burn Down Charts

Progress visualization of remaining work versus time; used in daily coordination.

15.2.7 Sprint Retrospective

Process inspection and adaptation; action items carried into the next planning session.

15.2.8 Individual Status Reports

Per-member updates for course reporting requirements.

15.2.9 Engineering Notebooks

Design decisions, experiments, and test notes (including Klipper/fswebcam capture and optional streamer URL findings and provisioning runs).

15.3 Closeout Materials

15.3.1 System Prototype

Working Vision Console with PostgreSQL, Firebase auth, optional CV, and at least one provisioned Raspberry Pi capture path.

15.3.2 Project Poster

CoE Innovation Day poster summarizing problem, approach, edge integration, and results.

15.3.3 Web Page

Project summary page for stakeholders and recruitment/demo context.

15.3.4 Demo Video

End-to-end walkthrough: register Pi, heartbeat online, Take Picture, view capture, optional CV.

15.3.5 Source Code

Monorepo application code, migrations, `cv-backend`, and reference `agrihome_agent` package.

15.3.6 Source Code Documentation

Implementation guide, CV pipeline notes, Raspberry Pi/Klipper ops docs (`RASPBERRY_PI_KLIPPER.md`), and edge security notes.

15.3.7 Hardware Schematics

Rack/fixture sketches and Pi/Klipper networking notes as applicable.

15.3.8 CAD Files

Fixture designs produced in makerspace workflows.

15.3.9 Installation Scripts

Environment templates, migration commands, agent register/run instructions.

15.3.10 User Manual

Operator guide for Console navigation, Devices page, tray Pi panel, and schedule destinations.

References

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, 2016.

[2] Surveyed CNN architectures (ResNet, VGG, MobileNet, EfficientNet) for tomato leaf disease detection in the plant-pathology deep learning literature (2018–2024).

[3] U-Net / Mask R-CNN leaf segmentation approaches for isolating plant tissue prior to classification.

[4] Agri-Home/klipper companion firmware and `camera-macros/save_image.sh` (`fswebcam`), used for AgriHome edge capture.

[5] AgriHome, *System Requirements Specification*, CSE 4316 / updated Summer 2026.

[6] AgriHome, *Architectural Design Specification*, CSE 4316, Spring 2026.

[7] Vercel, Inc., "Next.js Documentation," `https://nextjs.org/docs`.